UNIT-V

Transaction Management

Transaction Management:

- Transaction is an action or task used to perform one or more DBMS operations.

OR

- A transaction is a logical unit of work used to perform one or more DBMS tasks.

Example -ATM transaction steps.

- Transaction Start.
- Insert your ATM card.
- Select language for your transaction.
- Select Savings Account option.
- Enter the amount you want to withdraw.
- Enter your secret pin.
- Wait for some time for processing.
- Collect your Cash.
- Transaction Completed.

Three operations can be performed in a transaction as follows.

- Read/Access data (R).
- Write/Change data (W).
- Commit.

Let's take an example of a simple transaction. Suppose a bank employee transfers Rs 500 from A's account to B's account.

A's Account

Open_Account(A)

Old_Balance = A.balance(if balance=5000)(read)

New_Balance = Old_Balance – 500(5000-500)

A.balance = New_Balance (write)(4500)

Close_Account(A)

B's Account

Open_Account(B)

Old_Balance = B.balance (read)(if balance=3000)

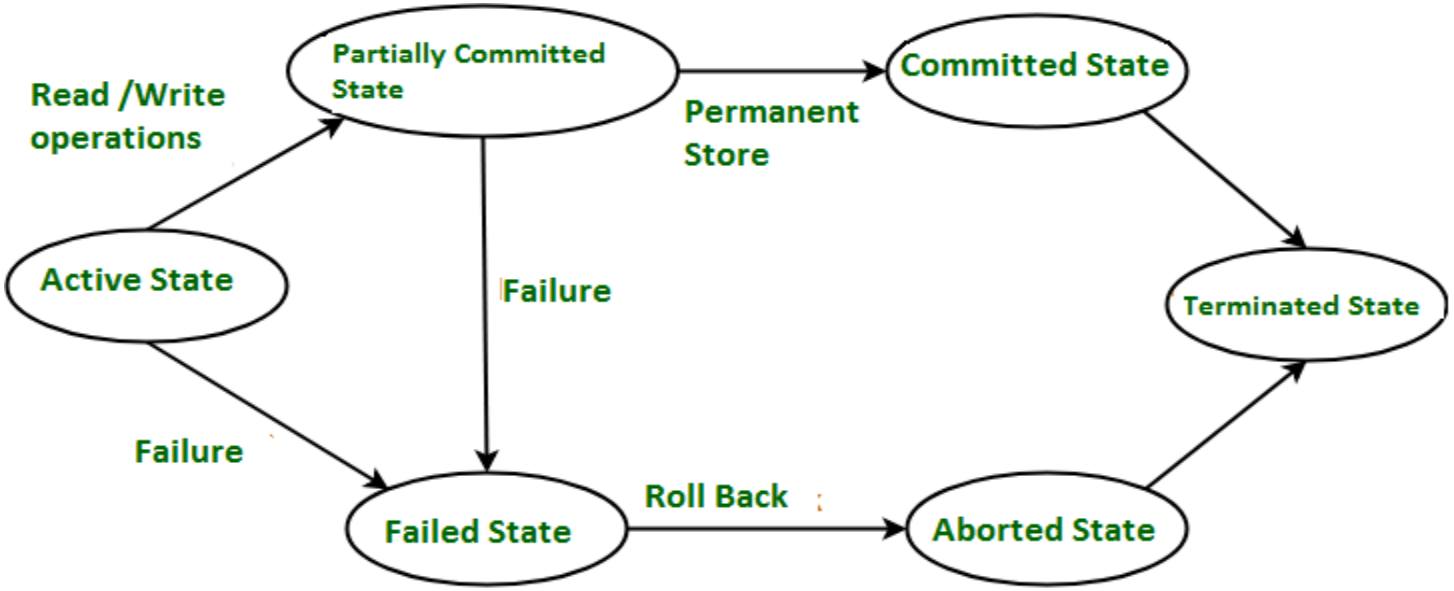
New_Balance = Old_Balance + 500(3000+500)

B.balance = New_Balance (write)(3500)

Close_Account(B)

Transaction states

- States through which a transaction goes during its lifetime.
- These are the states which tell about the current state of the Transaction and also tell how we will further do processing we will do on the transactions.
- These states govern the rules which decide the fate of the transaction whether it will commit or abort.



Transaction States in DBMS

1.Active State:

It is the first state of transaction. If a transaction is executed then it is called as active state

Ex: Any insertion, update or deletion of records is successfully executed.

2.Partially committed state:

If the transaction is successfully executed and if it is not permanently stored in the database then it is in partially committed state.

3.Failed State :

When any instruction of the transaction fails it goes to "failed state" or if failure occurs in making permanent change of data on Data Base.

4.Aborted State :

After having any type of failure the transaction goes from "failed state" to "aborted state" and in before states the changes are only made to local buffer or main memory and hence these changes are deleted or rollback.

5.Committed Stage :

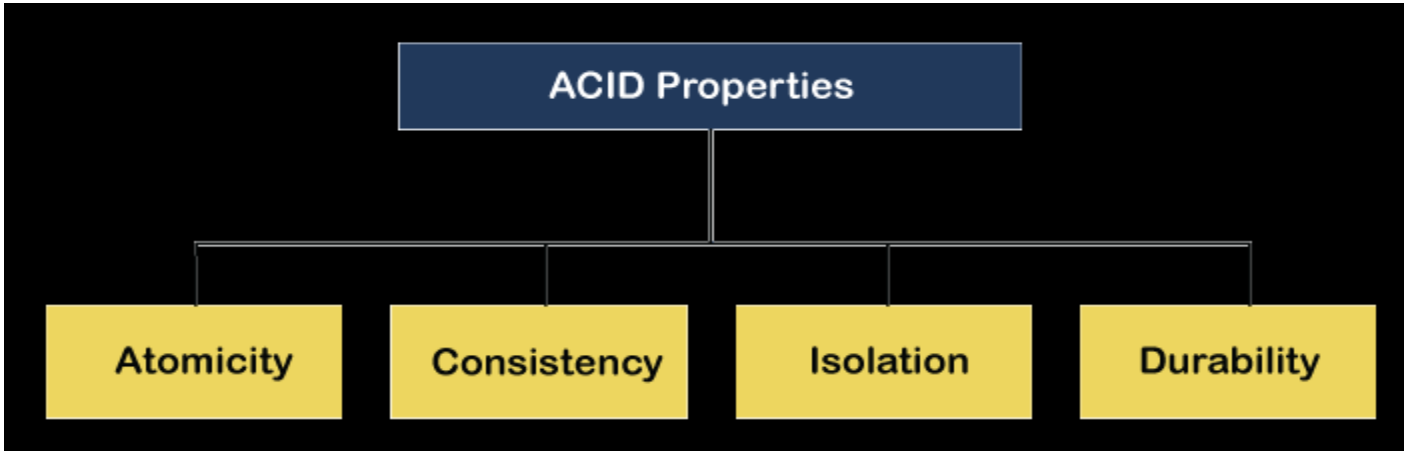
It is the stage when the changes are made permanent on the Data Base and transaction is complete and therefore terminated in "terminated state".

6.Terminated State :

If there is any roll back or the transaction come from "committed state" then the system is consistent and ready for new transaction and the old transaction is terminated.

Properties of Transaction or ACID properties:

To maintain consistency in database before or after the transaction some properties must be followed. Those properties of transaction are known as ACID properties.



Atomicity:

- If A Transaction can takes place(successful) or it can never happen(failed) then it justifies atomic property
- There should not any middle way(means amount deducted from A's account but not credited to B)

Example:

- The following transaction transfers 20 dollars from Alice's bank account to Bob's bank account. If any of the instructions fail, the entire transaction should abort and rollback.
- If the transaction fails after completion of **T1** but before completion of **T2**.(say, after **write(X)** but before **write(Y)**), then amount has been deducted from **X** but not added to **Y**.

Before: X : 500	Y: 200
Transaction T	
T1	**T2**
Read (X) X: = X − 100 Write (X)	Read (Y) Y: = Y + 100 Write (Y)
After: X : 400	Y : 300

Consistency

- A database is initially in a consistent state, and it should remain consistent after every transaction.
- Suppose that the transaction in the previous example fails after Write(A_b) and the transaction is not rolled back; then, the database will be inconsistent as the sum of Alice and Bob's money, after the transaction, will not be equal to the amount of money they had before the transaction.

Isolation

- If the multiple transactions are running concurrently, they should not be affected by each other; i.e., the result should be the same as the result obtained if the transactions were running sequentially.

Durability

- Changes that have been committed to the database should remain even in the case of software and hardware failure. For instance, if Bob's account contains $120, this information should not disappear upon hardware or software failure.

Implementation of atomicity and durability:

Atomicity-definition

Durability-definition

- The recovery management database system supports atomicity and durability with some schemes
- One of the scheme is "shadow-copy" scheme.
- Making multiple copies of the database is known as shadow copy.
- Shadow copy assumes only one transaction is active at a time.
- Shadow copy assumes that a database is simply a file that is stored in a disk.
- Shadow copy used a pointer called db-pointer which is maintained in the disk.
- Db-pointer points the current database in the disk.

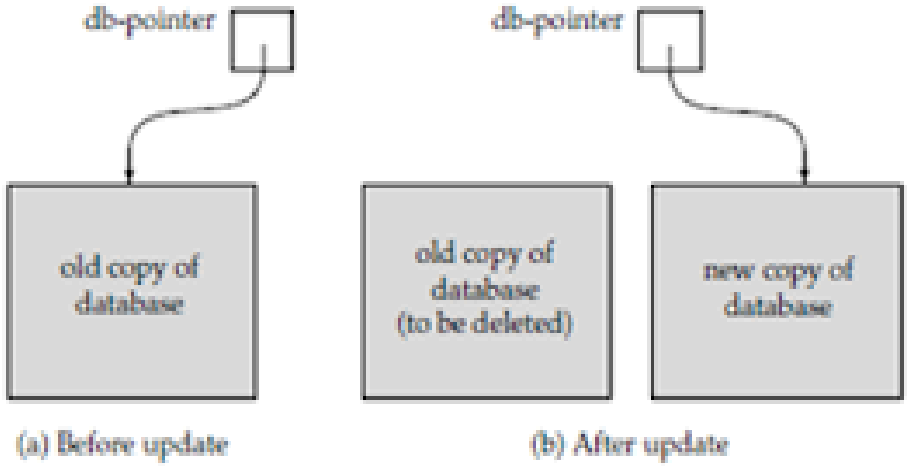


Figure 15.2 Shadow-copy technique for atomicity and durability.

Concurrent Executions:

Concurrent execution is a process of performing or executing all the transactions concurrently or simultaneously in a system.

Advantages:

1.No waiting time

2.Throughput or effective utilization of resources

Some of the problems of concurrent executions are

1.Write-write conflict(w-w conflict)

2.Read-write conflict(R-w conflict)

3.Write-read conflict(W-R conflict)

1.Write-write conflict:

Example: let's imagine A=100

Time	Transaction T1	Transaction T2
t1	Read(A)	
t2	A=A-50	
t3		Read(A)
t4		A=A+50
t5	Write(A)	
t6		Write(A)

2.Read-write conflict(R-w conflict)

If a transaction T_1 reads a value of data item twice and the data item is changed by another transaction T_2 in between the two read operation. Hence T_1 access two different values for its two read operation of the same data item.

Time	Transaction T1	Transaction T2
t1	Read(A)	
t2		Read(A)
t3		A=A+30
t4		Write(A)
t5	Read(A)	

Write-Read conflict:

- This type of problem occurs when one transaction T1 updates a data item of the database, and then that transaction fails due to some reason, but its updates are accessed by some other transaction.
- **Example:** Let's take the value of A is 100

Time	Transaction T1	Transaction T2
t1	Read(A)	
t2	A=A+20	
t3	Write(A)	
t4		Read(A)
t5		A=A+30
t6		Write(A)
t7	Write(B)	

Serializability:

- Serializability is a concept that helps to identify which non-serial schedules are correct and will maintain the consistency of the database.
- It relates to the isolation property of transaction in the database.
- Serializability is the concurrency scheme where the execution of concurrent transactions is equivalent to the transactions which execute serially.
- Serializability is a property where a transaction only starts its execution when another transaction has already completed its execution.
- However, a non-serial schedule of transactions needs to be checked for Serializability.

Testing of Serializability

- To test the serializability of a schedule, we can use the serialization graph.(Precedence graph)

- Suppose, a schedule S.
- For schedule S, construct a graph called as a precedence graph.
- It has a pair G = (V, E), where E consists of a set of edges, and V consists of a set of vertices.
- The set of vertices contain all the transactions participating in the S schedule.
- The set of edges contains all edges Ti ->Tj for which one of the following three conditions satisfy:
- Create a node Ti → Tj if Ti transaction executes write (Q) before Tj transaction executes read (Q).
- Create a node Ti → Tj if Ti transaction executes read (Q) before Tj transaction executes write (Q).
- Create a node Ti → Tj if Ti transaction executes write (Q) before Tj transaction executes write (Q).

Example:

Time	Transaction T1	Transaction T2
T1	Read(A)	
T2		Read(A)
T3		Write(A)
T4	A=A+50	
T5	Write(A)	

G=(v,E)

V={T1,T2}

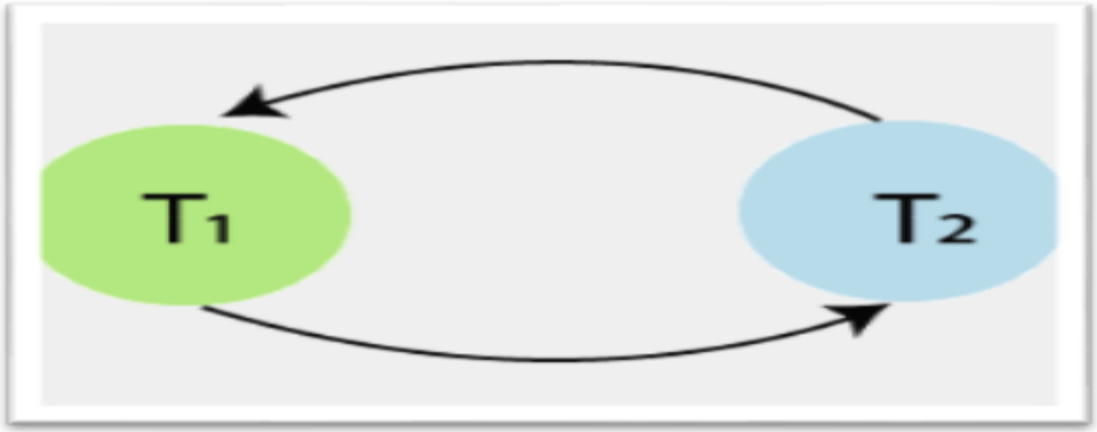
IN time T1,Transaction T1 reads A,which was subsequenty read by T2

T1->T2

T2->T1

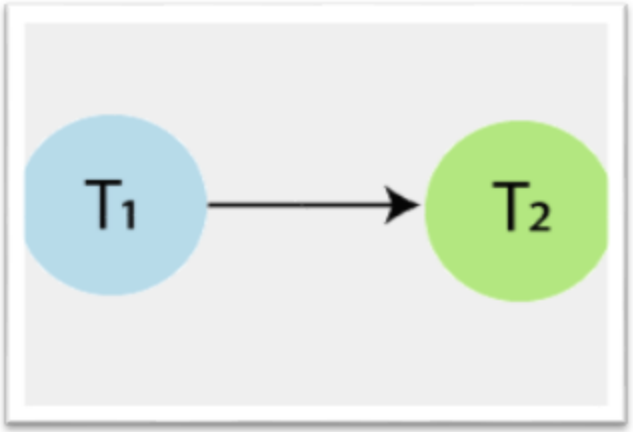
Serializability Graph for above example

Fig: Serializability graph

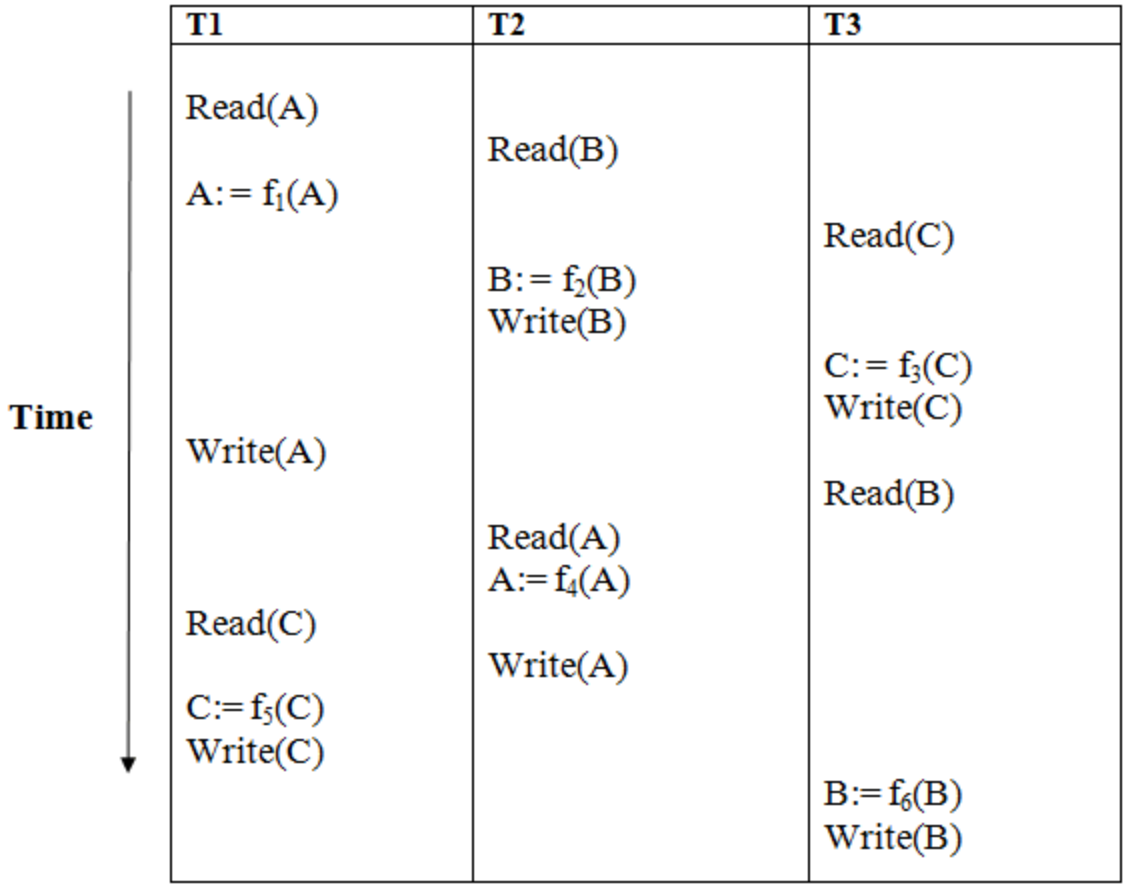


Example:2

Time	Transaction T1	Transaction T2
t1	Read(A)	
t2	A=A+50	
t3	Write(A)	
t4		Read(A)
t5		A+A+100
t6		Write(A)

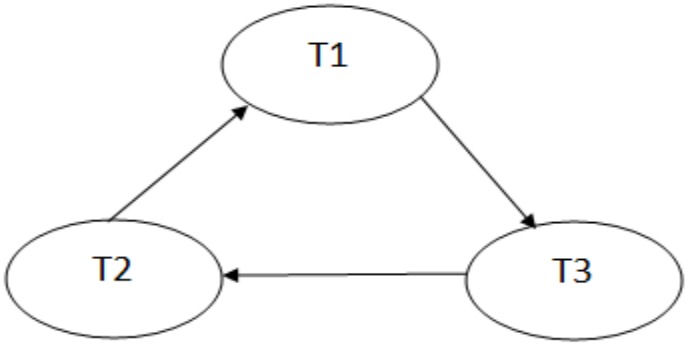


Example:3



Schedule S1

Read(A): In T1, no subsequent writes to A, so no new edges
Read(B): In T2, no subsequent writes to B, so no new edges
Read(C): In T3, no subsequent writes to C, so no new edges
Write(B): B is subsequently read by T3, so add edge T2 → T3
Write(C): C is subsequently read by T1, so add edge T3 → T1
Write(A): A is subsequently read by T2, so add edge T1 → T2
Write(A): In T2, no subsequent reads to A, so no new edges
Write(C): In T1, no subsequent reads to C, so no new edges
Write(B): In T3, no subsequent reads to B, so no new edges



Two types of serializability

1.View serializability

In view-serializability, the transactions tend to read the data that are committed (final-write) by another transaction or transaction itself and thus there won't be any issues in the data accuracy.

2.Conflict serializability

In conflict serializability, there is one write operation in one of the transactions which might affect the value in the database that is being read by another transaction thus resulting in false values being read. This is referred as conflict-serializability.

Recoverability:

- A transaction may not execute completely due to hardware failure, system crash or software issues.
- In that case, we have to roll back the failed transaction.
- But some other transaction may also have used values produced by the failed transaction.
- So we have to roll back those transactions as well.

Recoverable Schedules:

- Schedules in which transactions commit only after all transactions whose changes they read commit are called recoverable schedules.
- In other words, if some transaction T_j is reading value updated or written by some other transaction T_i, then the commit of T_j must occur after the commit of T_i.

- Example:

Let the schedule be S

T1	T2
R(A)	
W(A)	
	W(A)
	R(A)
Commit	
	Commit

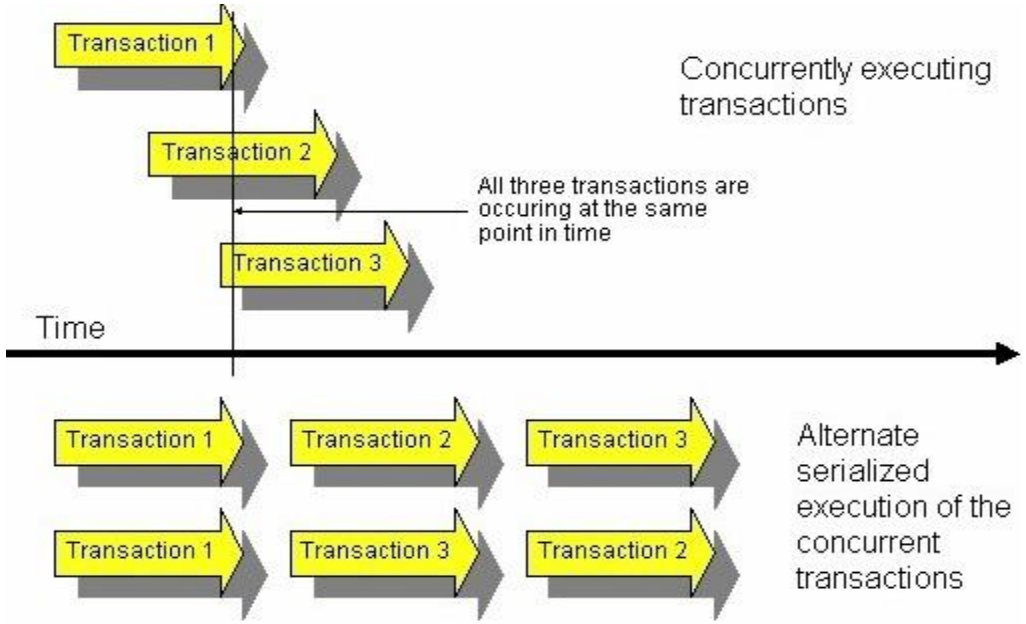
This is a recoverable schedule since T_1 commits before T_2, that makes the value read by T_2 correct.

Non recoverable Schedule:

T1	T2
R(A)	
W(A)	
	W(A)
	R(A)
failure	
	Commit
commit	

Implementation of isolation

- Isolation is a property where one transaction cannot affect the other transaction in a concurrent execution
- Finds the correctness of the data as well as data integrity.
- Avoids conflicts as much as possible
- Needs to maintain data consistency



Concurrency control & Recovery System

Concurrency control:

- **Concurrency Control** in Database Management System is a procedure of managing simultaneous operations without conflicting with each other.
- All transactions can be executed concurrently but shouldn't violate data integrity.
- When two or more users reads the data concurrently no problem exists.
- But DBMS is a combination of read and write operations where the actual problem exists.

Problems of concurrent transactions:

1.Loss of data

2.W-W conflict

3.R-W conflict

4.W-R conflict

Uses of concurrency control:

1.To avoid conflicts

2.To provide correctness of data

3.Provides data integrity

4.Should provide consistency in data

Concurrency control protocols:

Some Concurrency control protocols are used to control the concurrent executions. Some of them are:

1.Lock-based protocol

2.Time-stamp protocols

3.Validation protocols

1.Lock-based protocol

- Lock based protocol is a technique where the transaction begins read/write operations when it acquires lock
- Lock based protocols help to eliminate the concurrency problem in DBMS for simultaneous transactions by locking or isolating a particular transaction to a single user.
- A lock is a data variable which is associated with a data item

- All lock requests were handled by concurrency control manager and the transaction should begin only once the lock is granted

Binary lock: Binary lock on a data item can either locked or unlocked states.

Shared/Excusive lock:(S)

A shared lock is also called a Read-only lock. With the shared lock, the data item can be shared between transactions. This is because you will never have permission to update data on the data item.(lock-S instruction).

Exclusive lock:(X)

- In Exclusive Lock, a data item can be read as well as written. This is exclusive to only one transaction and can't be held concurrently on the same data item. X-lock is requested using lock-x instruction. Transactions will be given access to the data item after finishing the 'write' operation by first transaction.
- If second transaction wants to perform read/write operation simultaneously exclusive lock will prevent that operation.

Time-Stamp protocols:

- The Timestamp Ordering Protocol is used to order the transactions based on their Timestamps.
- The priority of the older transaction is higher that's why it executes first.
- The timestamp of the transaction uses system time or logical counter.
- Timestamp is used to serialize the execution of concurrent transactions. The Timestamp-based protocol ensures that every conflicting read and write operations are executed in a timestamp order.

Example:

- Suppose there are three transactions T1, T2, and T3.
- T1 has entered the system at time 0010 time
- T2 has entered the system at 0020
- T3 has entered the system at 0030
- Priority will be given to transaction T1, then transaction T2 and lastly Transaction T3.

Validation based protocol:

- **Validation based Protocol** in DBMS also known as Optimistic Concurrency Control Technique to control the conflicts in concurrent transactions.
- In this protocol, the local copies of the transaction data are updated rather than the data itself.

The Validation based Protocol is performed in the following three phases:

- Read Phase
- Validation Phase
- Write Phase

Read Phase

- In the Read Phase, the data values from the database can be read by a transaction but the write operation or updates are only applied to the local data copies, not the actual database.

Validation Phase

- In Validation Phase, the data is checked to ensure that there is no violation of serializability while applying the transaction updates to the database.

Write Phase

- In the Write Phase, the updates are applied to the database if the validation is successful, else; the updates are not applied, and the transaction is rolled back.

Crash Recovery:

- *Crash recovery* is the process by which the database is moved back to a consistent and usable state.
- This is done by rolling back incomplete transactions and completing committed transactions that were still in memory when the crash occurred.
- If the database or the database manager fails, the database can be left in an inconsistent state.
- A crash recovery operation must be performed in order to roll back the partially completed transactions and to write to disk the changes of completed transactions that were previously made only in memory.

- Some conditions when crash occurs

1.System stops responding

2.When the server doesn’t respond

3.Transaction fails due to insufficient data

4. A power failure on the machine, causing the database manager and the database partitions on it to go down.

5.A hardware failure such as memory, disk, CPU, or network failure.

- If you want crash recovery to be performed automatically by the database manager, enable the automatic restart (**autorestart**) database configuration parameter by setting it to ON.

Remote back up system:

- Remote back up systems helps in allowing the transaction to continue even if the primary site is crashed or destroyed.
- Back site will be automatically alerted when the primary site fails and it acts as primary site till the primary site is recovered or repaired.
- Some times we don't even know that the back up site is running.
- Back up site will detects the failure automatically.

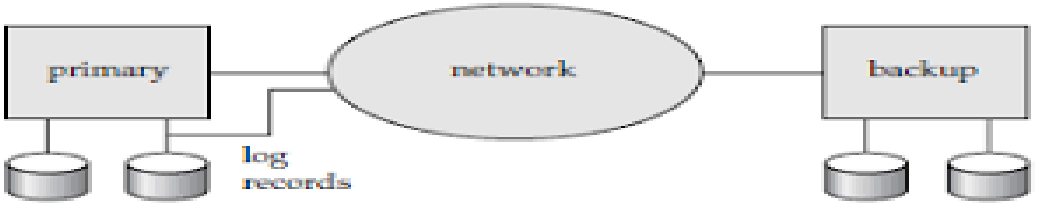


Figure 17.10 Architecture of remote backup system.

Storage and Indexing

Storage:

- A database system provides an ultimate view of the stored data. However, data in the form of bits, bytes get stored in different storage devices.
- Describes how the data is stored in the database.

Ex:hardsisk,magnetic disk, magnetic tapes, cache memory

Indexing:

What is Indexing?

- **Indexing** is a data structure technique which allows you to quickly retrieve records from a database file. An Index is a small table having only two columns.

Index structure:

- Indexes can be created using some database columns.

Search key	Reference number or pointer

Types of indexing:

1.Primary index

2.Secondary index

3.Clustered index

Primary index:

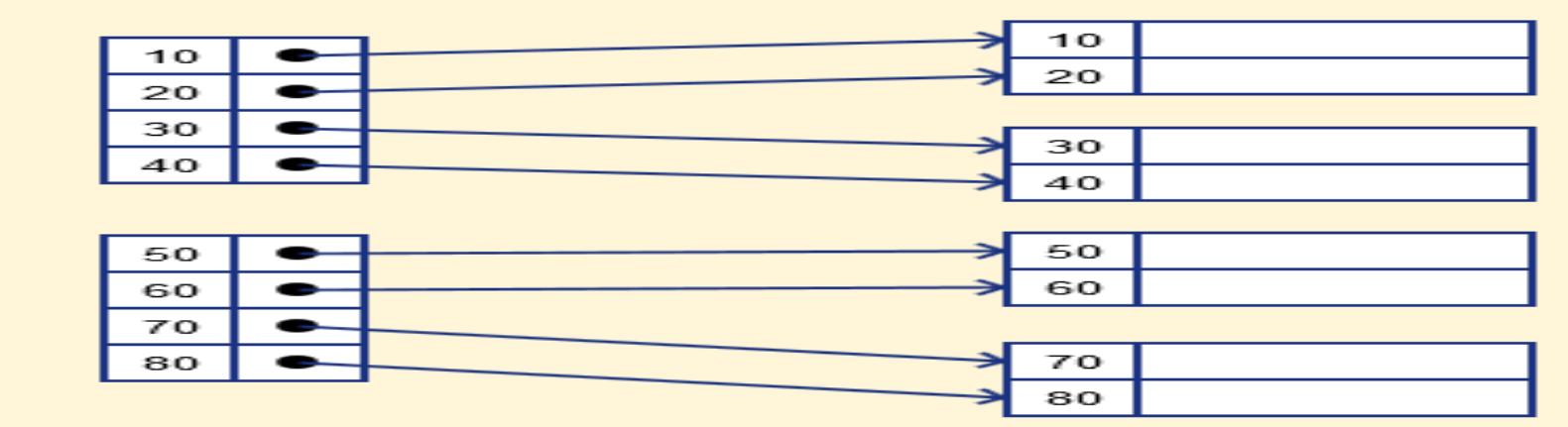
- If the index is created on the basis of the primary key of the table, then it is known as primary indexing. These primary keys are unique to each record .

Types of primary indexing:

- The primary Indexing in DBMS is also further divided into two types.
- Dense Index
- Sparse Index

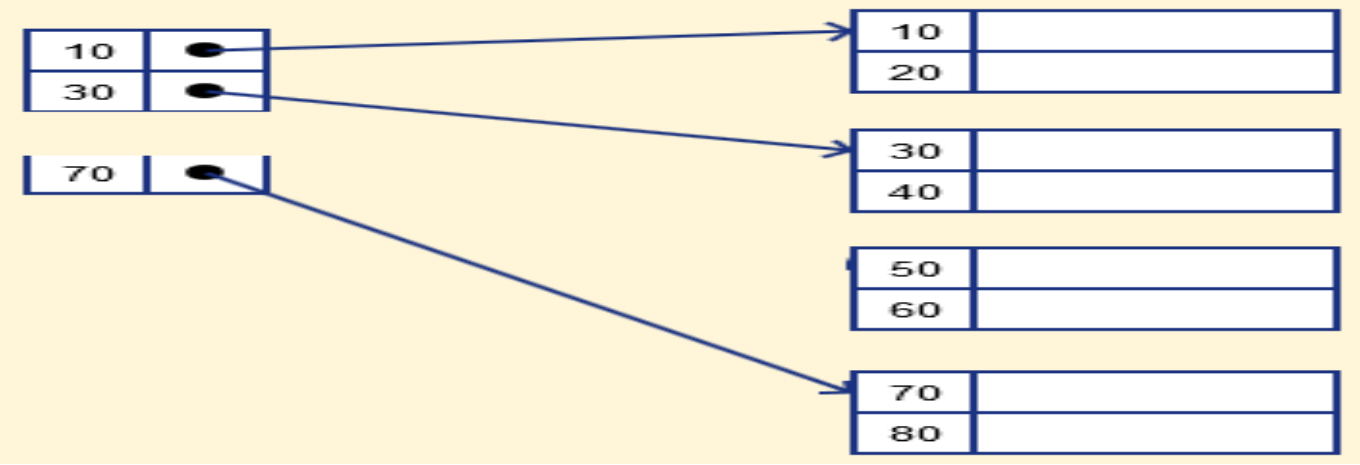
Dense Index

- In a dense index, a record is created for every search key valued in the database. This helps you to search faster but needs more space to store index records.



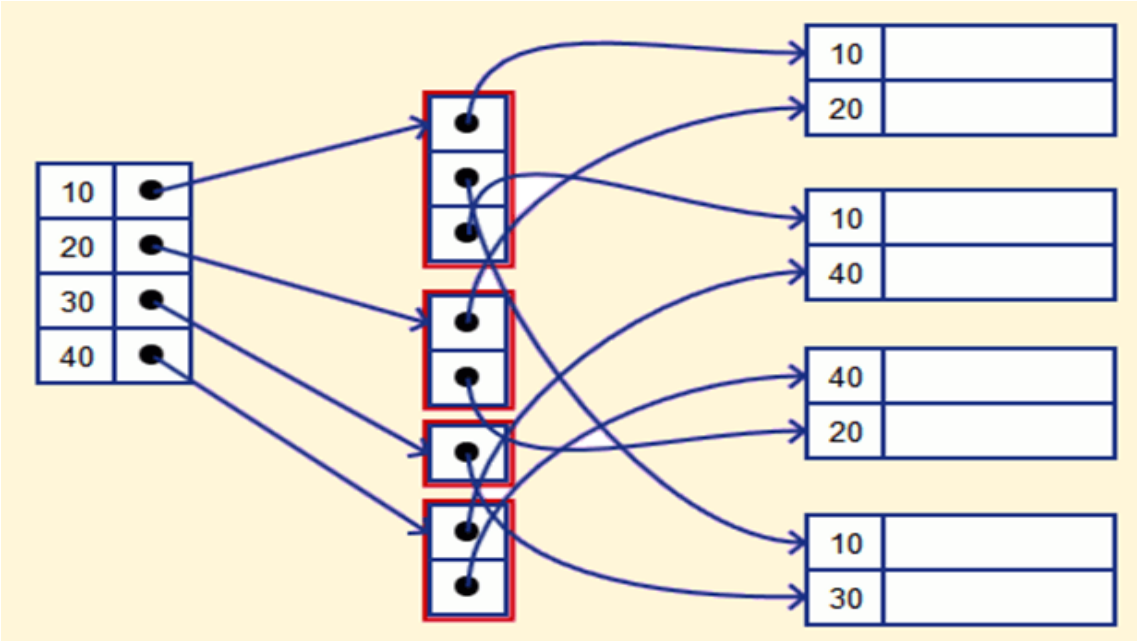
Sparse Index

- It is an index record that appears for only some of the values in the file. Sparse Index helps you to resolve the issues of dense Indexing in DBMS. In this method of indexing technique, a range of index columns stores the same data block address, and when data needs to be retrieved, the block address will be fetched.
- However, sparse Index stores index records for only some search-key values. It needs less space, less maintenance overhead for insertion, and deletions but It is slower compared to the dense Index for locating records.



Secondary Index

The secondary Index in DBMS can be generated by a field which has a unique value for each record, and it should be a candidate key.



Clustering Index

Group two or more columns to get the unique values and create an index which is called clustered Index. This also helps you to identify the record faster.

Example:

- Let's assume that a company recruited many employees in various departments. In this case, clustering indexing in DBMS should be created for all employees who belong to the same dept.

Multilevel Index:

- Multilevel Indexing in Database is created when a primary index does not fit in memory. In this type of indexing method, you can reduce the number of disk accesses to short any record and kept on a disk as a sequential file and create a sparse base on that file.

Tree structured indexing:

Tree-structured indexing techniques support both range searches and equality searches.

1.Indexed sequential Access method(ISAM)

2.B+ trees(Dynamic tree structure)

Indexed Sequential access method;(ISAM)

An Indexed Sequential Access Method (ISAM) is a file management technology developed by IBM and focused on fast retrieval of records which are maintained in the sort order with the help of an index.

- When data are being stored in an Indexed Sequential Access Method, they are entered sequentially.
- Data modification to the record did not require any changes to other records or indexes.
- ISAM can be used for searching equality or range values

Advantage:

1.Easy to understand

2.Less expensive

3.Fast accessing the data

4.It supports range retrieval or equality

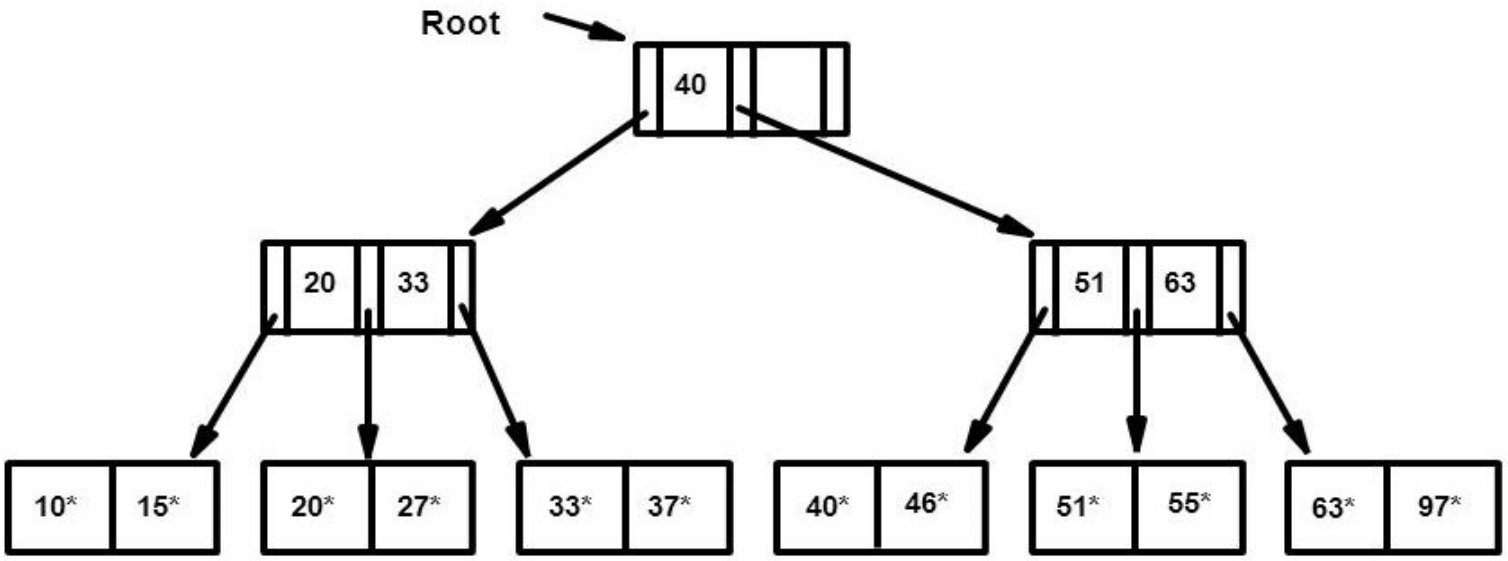
Disadvantage:

1.As it is static it requires more space.

2.More time for accessing when occupies more space

Example:

1. No of pointers is 1 greater than the search key.
2. Searching can be done only in leaf nodes.
3. Nodes are fixed during the creation of a tree.
4. Values assigned to the leaf node are fixed and will not change when any insertion or deletion occurs.



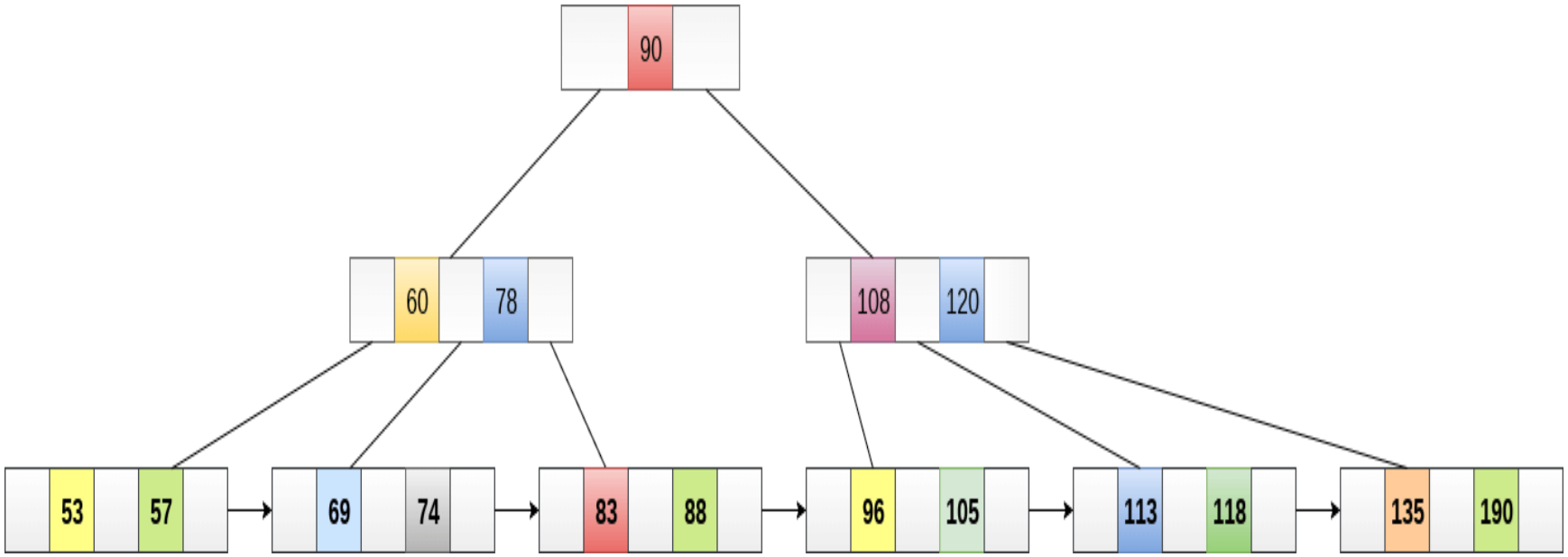
B+ trees:(dynamic tree structure)

- The B+ Trees are extended version of B-Trees. This tree supports better insertion, deletion and searching over B-Tree.
- B-trees, the keys and the record values are stored in the internal as well as leaf nodes. In B+ tree records, can be stored at the leaf node, internal nodes will store the key values only. The leaf nodes of the B+ trees are also linked like linked list
- B+ Tree are used to store the large amount of data which can not be stored in the main memory. Due to the fact that, size of main memory is always limited, the internal nodes (keys to access records) of the B+ tree are stored in the main memory whereas, leaf nodes are stored in the secondary memory.

Advantages over B-Tree

- Records can be fetched in equal number of disk accesses
- Height of the tree remains balanced, and less as compared to B-Trees
- As the leafs are connected like linked list, we can search elements in sequential manner also
- Keys are used for indexing
- The searching is faster, as data are stored at leaf level only.

Example of B+ tree



This supports basic operations like searching, insertion, deletion. In each node, the item will be sorted. The element at position i has child before and after it. So children sored before will hold smaller values, and children present at right will hold bigger values.